

1.3. NumPy massivlarida amallar (Operations in NumPy Array)

NumPy massivlari ma'lumotlar bilan samarali ishlash (data manipulation) imkonini beruvchi to'rtta asosiy amal turini taqdim etadi:

- **Binar amallar (Binary Operations)** – Elementlar bo'yicha solishtirish (masalan, kattaroq, teng) va “VA” (AND) yoki “YOKI” (OR) kabi mantiqiy operatsiyalarni bajaradi.
- **Matematik amallar (Mathematical Operations)** – `sum()` (yig'indi), `mean()` (o'rtacha qiymat) va `sqrt()` (kvadrat ildiz) kabi funksiyalar tezkor va samarali sonli hisob-kitoblarni amalga oshirish imkonini beradi.
- **Satr amallari (String Operations)** – Massivlar ichidagi matnlarni birlashtirish (concatenation), bosh harflarga o'tkazish (uppercasing) va almashtirish (replacing) kabi amallar uchun `np.char` funksiyalaridan foydalaniladi.
- **Arifmetik amallar (Arithmetic Operations)** – Massivlarda elementlar bo'yicha qo'shish, ayirish, ko'paytirish va bo'lish amallarini tezkor bajarishni ta'minlaydi.

1.3.1. Binar amallar (Binary Operations)

Binar amallar (Binary operations) bitlar ustida ishlaydi va bitma-bit operatsiyalarni amalga oshiradi. Oddiy qilib aytganda, binar amal — bu yangi qiymat hosil qilish uchun ikkita qiymatni birlashtirish qoidasidir.

```
numpy.bitwise_and()
```

Ushbu funksiya ikkita massiv elementlarining bitma-bit “VA” (AND) amalini hisoblash uchun ishlatiladi. Funksiya kirish massivlaridagi butun sonlarning asosiy binar ko'rinishi (ikkilik sanoq tizimidagi ko'rinishi) ustida amallarni bajaradi.

1-misol: Oddiy sonlar ustida qo'llash

```
import numpy as np

# Kirish sonlari
son1 = 10 # Binar ko'rinishi: 1010
son2 = 11 # Binar ko'rinishi: 1011
```

```
print("1-son:", son1)
print("2-son:", son2)

# Bitma-bit AND amali
natija = np.bitwise_and(son1, son2)
print("10 va 11 ning bitwise_and natijasi:", natija)
```

```
1-son: 10
2-son: 11
10 va 11 ning bitwise_and natijasi: 10
```

2-misol: Massivlar ustida qo'llash

```
# Kirish massivlari
massiv1 = [2, 8, 125]
massiv2 = [3, 3, 115]

print("1-massiv:", massiv1)
print("2-massiv:", massiv2)

# Elementlar bo'yicha bitma-bit AND amali
natija_massiv = np.bitwise_and(massiv1, massiv2)
print("bitwise_and natijasi:", natija_massiv)
```

```
1-massiv: [2, 8, 125]
2-massiv: [3, 3, 115]
bitwise_and natijasi: [ 2  0 113]
```

Izoh: Funksiya har bir elementni o'zaro solishtiradi. Masalan, ikkinchi juftlik elementlar uchun:

- 8 ning binar shakli: 1000
- 3 ning binar shakli: 0011
- Ular o'rtasida **AND** amali bajarilganda hech qanday bit mos kelmaydi, shuning uchun natija 0 bo'ladi.

numpy.bitwise_or()

Ushbu funksiya ikkita massiv elementlarining bitma-bit “YOKI” (**OR**) amalini hisoblash uchun ishlatiladi. Funksiya kirish massivlaridagi butun sonlarning asosiy binar (ikkilik) ko'rinishi ustida amallarni bajaradi.

1-misol: Oddiy sonlar ustida qo'llash

```
# Kirish sonlari
son1 = 10 # Binar ko'rinishi: 1010
son2 = 11 # Binar ko'rinishi: 1011

print("1-son:", son1)
print("2-son:", son2)

# Bitma-bit OR amali
natija = np.bitwise_or(son1, son2)
print("10 va 11 ning bitwise_or natijasi:", natija)
```

```
1-son: 10
2-son: 11
10 va 11 ning bitwise_or natijasi: 11
```

2-misol: Massivlar ustida qo'llash

```
# Kirish massivlari
massiv1 = [2, 8, 125]
massiv2 = [3, 3, 115]

print("1-massiv:", massiv1)
print("2-massiv:", massiv2)

# Elementlar bo'yicha bitma-bit OR amali
natija_massiv = np.bitwise_or(massiv1, massiv2)
print("bitwise_or natijasi:", natija_massiv)
```

```
1-massiv: [2, 8, 125]
2-massiv: [3, 3, 115]
bitwise_or natijasi: [ 3 11 127]
```

Mantiqiy tahlil: Nima uchun 8 va 3 dan 11 hosil bo'ldi?

Bitma-bit **OR (YOKI)** amali bajarilishi uchun solishtirilayotgan bitlarning kamida bittasi **1** bo'lishi kifoya.

$$\begin{array}{rcl}
 8 & = & 1000_2 \\
 3 & = & 0011_2 \\
 \hline
 \text{OR} & = & 1011_2 \rightarrow (11_{10})
 \end{array}$$

Izoh:

1. Oxirgi bitlar: **0** va **1** → natija **1**.
2. Undan oldingi: **0** va **1** → natija **1**.

3. Undan oldingi: 0 va $0 \rightarrow$ natija 0 .

4. Birinchi bitlar: 1 va $0 \rightarrow$ natija 1 .

Natijada 1011 hosil bo'ladi, bu o'nlik sanoq tizimida 11 ga teng.

`numpy.bitwise_xor()`

Ushbu funksiya ikkita massiv elementlarining bitma-bit “**istisno qiluvchi YOKI**” (XOR) amalini hisoblash uchun ishlatiladi. Funksiya kirish massivlaridagi butun sonlarning asosiy binar (ikkilik) ko'rinishi ustida amallarni bajaradi.

1-misol: Oddiy sonlar ustida qo'llash

```
# Kirish sonlari
son1 = 10 # Binar ko'rinishi: 1010
son2 = 11 # Binar ko'rinishi: 1011

print("1-son:", son1)
print("2-son:", son2)

# Bitma-bit XOR amali
natija = np.bitwise_xor(son1, son2)
print("10 va 11 ning bitwise_xor natijasi:", natija)
```

1-son: 10

2-son: 11

10 va 11 ning bitwise_xor natijasi: 1

2-misol: Massivlar ustida qo'llash

```
# Kirish massivlari
massiv1 = [2, 8, 125]
massiv2 = [3, 3, 115]

print("1-massiv:", massiv1)
print("2-massiv:", massiv2)

# Elementlar bo'yicha bitma-bit XOR amali
natija_massiv = np.bitwise_xor(massiv1, massiv2)
print("bitwise_xor natijasi:", natija_massiv)
```

1-massiv: [2, 8, 125]

2-massiv: [3, 3, 115]

bitwise_xor natijasi: [1 11 14]

Mantiqiy tahlil: XOR qanday ishlaydi?

XOR (Exclusive OR) amali “farqli bo‘lsa 1, bir xil bo‘lsa 0” qoidasi asosida ishlaydi. Ya’ni, solishtirilayotgan bitlar har xil bo‘lgandagina natija **1** bo‘ladi.

Misol: 8 va 3 dan nima uchun 11 hosil bo‘ldi?

$$\begin{array}{r} 8 = 1000_2 \\ 3 = 0011_2 \\ \hline \text{XOR} = 1011_2 \rightarrow (11_{10}) \end{array}$$

Misol: 125 va 115 dan nima uchun 14 hosil bo‘ldi?

$$\begin{array}{r} 125 = 1111101_2 \\ 115 = 1110011_2 \\ \hline \text{XOR} = 0001110_2 \rightarrow (14_{10}) \end{array}$$

Izoh:

- Ikkala bit **1** bo‘lsa $\rightarrow 0$
- Ikkala bit **0** bo‘lsa $\rightarrow 0$
- Bittasi **1** va bittasi **0** bo‘lsa $\rightarrow 1$

`numpy.invert()`

Ushbu funksiya massiv elementlarining bitma-bit **teskarisini (Inversion)** hisoblash uchun ishlatiladi. U kirish massivlaridagi butun sonlarning binar ko‘rinishi ustida **"EMAS" (NOT)** mantiqiy amalini bajaradi.

Muhim jihati: Ishorali butun sonlar (signed integers) uchun funksiya **ikki qo‘shimcha (two’s complement)** qiymatni qaytaradi. Ikki qo‘shimcha tizimida manfiy sonlar mutlaq qiymatning (absolute value) teskarisiga birni qo‘shish orqali ifodalanadi. Oddiy qilib aytganda, butun sonlar uchun natija $-(x + 1)$ formulasiga muvofiq bo‘ladi.

1-misol: Oddiy son ustida qo‘llash

```
in_num = 10
print("Kirish soni:", in_num)

# Bitma-bit NOT amali
out_num = np.invert(in_num)
print("10 ning inversiyasi:", out_num)
```

Kirish soni: 10
10 ning inversiyasi: -11

2-misol: Massiv ustida qo'llash

```
in_arr = [2, 0, 25]
print("Kirish massivi:", in_arr)

# Massiv elementlari uchun inversiya
out_arr = np.invert(in_arr)
print("Inversiyadan keyingi natija:", out_arr)
```

Kirish massivi: [2, 0, 25]
Inversiyadan keyingi natija: [-3 -1 -26]

Mantiqiy tahlil: Nima uchun 10 dan -11 hosil bo'ldi?

Kompyuterda ishorali sonlar xotirada qanday saqlanishini tushunish natijani izohlashga yordam beradi:

1. 10 sonining binar ko'rinishi (8-bitda): 00001010
2. Bitma-bit invert (NOT) amali: 11110101
3. Ikki qo'shimcha tizimida: 11110101 soni manfiy sonni anglatadi. Uni o'nlik tizimga qaytarish uchun barcha bitlar teskarisiga aylantiriladi (00001010) va 1 qo'shiladi (00001011). Bu esa 11 ga teng, ishorasi manfiy bo'lgani uchun natija -11 bo'ladi.

Qisqa formula:

Har qanday butun x soni uchun `np.invert(x)` natijasi $-(x + 1)$ ga teng bo'ladi.

- Masalan: $2 \rightarrow -(2 + 1) = -3$
- Masalan: $0 \rightarrow -(0 + 1) = -1$
- Masalan: $25 \rightarrow -(25 + 1) = -26$

`numpy.left_shift()`

Ushbu funksiya butun sonning bitlarini chapga siljitish uchun ishlatiladi. Siljitish jarayoni birinchi massivdagi (`arr1`) sonlarning o'ng tomoniga ikkinchi massivda (`arr2`) ko'rsatilgan miqdorda nollarni qo'shish orqali amalga oshiriladi.

Sonlarning ichki ko'rinishi binar (ikkilik) formatda bo'lgani uchun, bu amal matematik jihatdan birinchi sonni 2^{arr2} ga ko'paytirish bilan barobardir. Masalan, agar

son **5** bo'lsa va biz uni **2 bit** chapga siljitmoqchi bo'lsak, natija $5 \cdot 2^2 = 20$ bo'ladi.

1-misol: Oddiy son ustida qo'llash

```
in_num = 5
bit_shift = 2

print("Kirish soni:", in_num)
print("Siljitish kerak bo'lgan bitlar soni:", bit_shift)

# Chapga siljitish amali
out_num = np.left_shift(in_num, bit_shift)
print("2 bit chapga siljitgandan keyin:", out_num)
```

Kirish soni: 5
Siljitish kerak bo'lgan bitlar soni: 2
2 bit chapga siljitgandan keyin: 20

2-misol: Massiv ustida qo'llash

```
in_arr = [2, 8, 15]
bit_shift = [3, 4, 5]

print("Kirish massivi:", in_arr)
print("Siljitish miqdori:", bit_shift)

# Massiv elementlarini mos ravishda siljitish
out_arr = np.left_shift(in_arr, bit_shift)
print("Natija massivi:", out_arr)
```

Kirish massivi: [2, 8, 15]
Siljitish miqdori: [3, 4, 5]
Natija massivi: [16 128 480]

Mantiqiy tahlil: Nima uchun 5 dan 20 hosil bo'ldi?

Keling, buni binar (ikkilik) tizimda ko'rib chiqamiz:

1. 5 sonining binar shakli: 00000101
2. **2 bit chapga siljitish:** Bitlarni chapga suramiz va o'ngdan ikkita 0 qo'shamiz:
00010100
3. **Hosil bo'lgan son:** 00010100 binar tizimda 20 ga teng.

Matematik tekshirish:

- $2 \cdot 2^3 = 2 \cdot 8 = 16$
- $8 \cdot 2^4 = 8 \cdot 16 = 128$

- $15 \cdot 2^5 = 15 \cdot 32 = 480$

Chapga siljitish amali asosan tezkor ko'paytirish sifatida dasturlashda keng qo'llaniladi.

`numpy.right_shift()`

Ushbu funksiya butun sonning bitlarini o'ngga siljitish uchun ishlatiladi. Kompyuterda sonlarning ichki ko'rinishi binar (ikkilik) formatda bo'lgani sababli, bu amal matematik jihatdan birinchi sonni (**arr1**) 2^{arr2} ga bo'lish (butun qismini olish) bilan barobardir.

Masalan, agar son **20** bo'lsa va biz uni **2 bit** o'ngga siljitmoqchi bo'lsak, natija $20/2^2 = 5$ bo'ladi.

1-misol: Oddiy son ustida qo'llash

```
in_num = 20
bit_shift = 2

print("Kirish soni:", in_num)
print("Siljitish kerak bo'lgan bitlar soni:", bit_shift)

# o'ngga siljitish amali
out_num = np.right_shift(in_num, bit_shift)
print("2 bit o'ngga siljitgandan keyin:", out_num)
```

```
Kirish soni: 20
Siljitish kerak bo'lgan bitlar soni: 2
2 bit o'ngga siljitgandan keyin: 5
```

2-misol: Massiv ustida qo'llash

```
in_arr = [24, 48, 16]
bit_shift = [3, 4, 2]

print("Kirish massivi:", in_arr)
print("Siljitish miqdori:", bit_shift)

# Massiv elementlarini mos ravishda o'ngga siljitish
out_arr = np.right_shift(in_arr, bit_shift)
print("Natija massivi:", out_arr)
```

```
Kirish massivi: [24, 48, 16]
Siljitish miqdori: [3, 4, 2]
Natija massivi: [3 3 4]
```

Mantiqiy tahlil: Nima uchun 20 dan 5 hosil bo'ldi?

Binar (ikkilik) tizimda jarayon quyidagicha ko'rinadi:

1. **20** sonining binar shakli: `00010100`
2. **2 bit o'ngga siljitish:** Barcha bitlarni 2 marta o'ngga suramiz. O'ng chekkadagi bitlar "tushib qoladi", chapdan esa nollar qo'shiladi: `00000101`
3. **Hosil bo'lgan son:** `00000101` binar tizimda **5** ga teng.

Matematik tekshirish:

- $24/2^3 = 24/8 = 3$
- $48/2^4 = 48/16 = 3$
- $16/2^2 = 16/4 = 4$

O'ngga siljitish amali dasturlashda sonni 2 ning darajalariga juda tez (optimizatsiya qilingan tarzda) bo'lish uchun ishlatiladi.

```
numpy.binary_repr(son, width=None)
```

Ushbu funksiya kiritilgan sonning ikkilik (binar) ko'rinishini **satr (string)** shaklida qaytarish uchun ishlatiladi.

Manfiy sonlar bilan ishlash xususiyati:

- Agar `width` (kenglik) parametri berilmasa, manfiy sonning oldiga oddiygina minus ("-") belgisi qo'yiladi.
- Agar `width` parametri ko'rsatilsa, sonning shu kenglikka nisbatan **ikki qo'shimcha (two's complement)** kodi qaytariladi.

Kompyuterlarda ishorali butun sonlarni ifodalashning eng keng tarqalgan usuli aynan ikki qo'shimcha tizimidir. Bunda manfiy son mutlaq qiymatning ikkilikdagi teskarisiga birni qo'shish orqali hosil qilinadi.

1-misol: Oddiy sonning ikkilik ko'rinishi

```
son = 10
print("Kirish soni:", son)

# 10 sonining ikkilik ko'rinishini olamiz
natija = np.binary_repr(son)
print("10 ning ikkilik ko'rinishi:", natija)
```

Kirish soni: 10

10 ning ikkilik ko'rinishi: 1010

2-misol: width parametri bilan ishlash (Musbat va manfiy sonlar)

```
# Musbat 5 va manfiy 8 sonlari
sonlar = [5, -8]

# 5 soni uchun (width ishlatmasdan va ishlatib)
print("5 sonining ikkilik ko'rinishi:")
print("width ishlatilmasa:", np.binary_repr(sonlar[0]))
print("width = 5 bo'lganda:", np.binary_repr(sonlar[0], width=5))

print("\n-8 sonining ikkilik ko'rinishi:")
# -8 uchun (width ishlatilmasa va ishlatib)
print("width ishlatilmasa:", np.binary_repr(sonlar[1]))
print("width = 5 bo'lganda (ikki qo'shimcha kodi):", np.binary_repr(sonlar[1]
```

5 sonining ikkilik ko'rinishi:

width ishlatilmasa: 101

width = 5 bo'lganda: 00101

-8 sonining ikkilik ko'rinishi:

width ishlatilmasa: -1000

width = 5 bo'lganda (ikki qo'shimcha kodi): 11000

Mantiqiy tahlil: Nima uchun -8 soni '11000' bo'ldi?

width=5 bo'lganda, manfiy son ikki qo'shimcha tizimi bo'yicha hisoblanadi:

1. 8 sonining 5-bitli ko'rinishi: 01000

2. Inversiya (teskarisiga aylantirish): 10111

3. 1 qo'shish: $10111 + 1 = 11000$

Bu usul dasturchilarga sonlarning xotiradagi haqiqiy bit ko'rinishini satr sifatida ko'rish imkonini beradi.

`numpy.packbits(myarray, axis=None)`

Ushbu funksiya binar qiymatlardan (0 va 1) tashkil topgan massiv elementlarini bitlar sifatida **uint8** (8-bitli ishorasiz butun son) turidagi massivga joylashtiradi (paketlaydi). Agar elementlar soni to'liq baytni (8 bitni) tashkil qilmasa, natija oxiriga nol bitlar qo'shish orqali to'ldiriladi.

Misol:

```
# 3 o'lchamli binar massiv yaratamiz
a = np.array([[[1, 0, 1],
               [0, 1, 0]],
              [[1, 1, 0],
               [0, 0, 1]]])

# Oxirgi o'q (axis=-1) bo'yicha elementlarni bitlarga joylashtiramiz
b = np.packbits(a, axis=-1)

print(b)
```

```
[[[160]
   [ 64]]
```

```
[[[192]
   [ 32]]]
```

Mantiqiy tahlil: Natijalar qayerdan keldi?

Ushbu funksiya qanday ishlashini tushunish uchun har bir kichik massivni ko'rib chiqamiz. `axis=-1` bo'lgani uchun u eng ichki qavslardagi 3 ta elementni oladi va ularni 8-bitli baytning boshiga qo'yadi, qolgan 5 ta bo'sh joyni esa nollar bilan to'ldiradi.

1. Birinchi element: `[1, 0, 1]`

- Bitlar: `1 0 1` + (5 ta nol) `0 0 0 0 0` = `10100000`
- Ikkilikdan o'nlikka: $2^7 + 2^5 = 128 + 32 = 160$

2. Ikkinchi element: `[0, 1, 0]`

- Bitlar: `0 1 0` + (5 ta nol) `0 0 0 0 0` = `01000000`
- Ikkilikdan o'nlikka: $2^6 = 64$

3. Uchinchi element: `[1, 1, 0]`

- Bitlar: `1 1 0` + (5 ta nol) `0 0 0 0 0` = `11000000`
- Ikkilikdan o'nlikka: $2^7 + 2^6 = 128 + 64 = 192$

4. To'rtinchi element: `[0, 0, 1]`

- Bitlar: `0 0 1` + (5 ta nol) `0 0 0 0 0` = `00100000`
- Ikkilikdan o'nlikka: $2^5 = 32$

Xulosa: `packbits` funksiyasi binar ma'lumotlarni xotirada 8 barobar ixchamroq saqlash imkonini beradi, chunki har bir 0 yoki 1 qiymati alohida bayt emas, balki baytning bitta biti sifatida saqlanadi.

`numpy.unpackbits(myarray, axis=None)`

Ushbu funksiya `uint8` (8-bitli ishorasiz butun son) turidagi massiv elementlarini qismlarga ajratib, ularni binar qiymatli (0 va 1 lardan iborat) massivga aylantiradi. `myarray` ning har bir elementi bitta bit maydoni deb qaraladi va u 8 ta alohida bitga yoyib chiqiladi.

Natijaviy massiv shakli:

- Agar `axis=None` bo'lsa, natija 1 o'lchamli (1-D) massiv bo'ladi.
- Agar o'q (`axis`) ko'rsatilsa, natijaviy massiv kirish massivi bilan bir xil shaklda bo'ladi, faqat ko'rsatilgan o'q bo'yicha bitlar yoyiladi.

Misol:

```
# uint8 turidagi ustun ko'rinishdagi massiv yaratamiz
a = np.array([[2], [7], [23]], dtype=np.uint8)

# Elementlarni 1-o'q (ustunlar yo'nalishi) bo'yicha binar ko'rinishga yoyamiz
b = np.unpackbits(a, axis = 1)

print(b)

[[0 0 0 0 0 0 1 0]
 [0 0 0 0 0 1 1 1]
 [0 0 0 1 0 1 1 1]]
```

Mantiqiy tahlil: Natijalar qanday hosil bo'ldi?

Har bir butun son 8-bitli xotira katakchasida qanday saqlangan bo'lsa, `unpackbits` uni aynan shunday ko'rinishda massiv elementlariga ajratadi:

1. 2 soni uchun:

- 2 ning ikkilik sanoq tizimidagi ko'rinishi: `10`
- 8-bitli formatda (chapdan nollar bilan to'ldirilgan): `00000010`
- Natija: `[0, 0, 0, 0, 0, 0, 1, 0]`

2. 7 soni uchun:

- 7 ning ikkilik ko'rinishi: `111`
- 8-bitli formatda: `00000111`

- Natija: `[0, 0, 0, 0, 0, 1, 1, 1]`

3. 23 soni uchun:

- 23 ning ikkilik ko‘rinishi: $16 + 4 + 2 + 1 = 10111$
- 8-bitli formatda: `00010111`
- Natija: `[0, 0, 0, 1, 0, 1, 1, 1]`

Xulosa: `unpackbits` funksiyasi `packbits` funksiyasining teskarisi bo‘lib, xotirada siqilgan holatdagi bitlarni qaytadan tahlil qilish (masalan, tasvirlarni qayta ishlash yoki binar ma'lumotlarni o‘qish) uchun juda qulaydir.

1.3.2. Matematik amallar (Mathematical Operations)

NumPy kutubxonasi matematik amallarning ulkan to‘plamini o‘z ichiga oladi. U standart trigonometrik funksiyalar, arifmetik amallar, kompleks sonlar bilan ishlash va boshqa ko‘plab imkoniyatlarni taqdim etadi.

Trigonometrik funksiyalar

NumPy radianlarda berilgan burchaklar uchun trigonometrik nisbatlarni elementlar bo‘yicha (element-wise) hisoblash uchun `sin()`, `cos()` va `tan()` kabi funksiyalarni taqdim etadi.

Quyidagi jadvalda NumPy-dagi asosiy trigonometrik funksiyalar va ularning tavsifi keltirilgan:

Funksiya	Tavsifi
<code>sin()</code>	Sinusni elementlar bo‘yicha hisoblaydi.
<code>cos()</code>	Kosinusni elementlar bo‘yicha hisoblaydi.
<code>tan()</code>	Tangensni elementlar bo‘yicha hisoblaydi.
<code>arcsin()</code>	Arksinus (teskari sinus) amali.
<code>arccos()</code>	Arkkosinus (teskari kosinus) amali.
<code>arctan()</code>	Arktangens (teskari tangens) amali.

Funksiya	Tavsifi
<code>arctan2()</code>	Ikki argumentli arktangens (kvadrantni to'g'ri tanlagan holda).
<code>degrees()</code> / <code>rad2deg()</code>	Burchaklarni radiandan darajaga o'tkazadi.
<code>radians()</code> / <code>deg2rad()</code>	Burchaklarni darajadan radianga o'tkazadi.
<code>hypot()</code>	To'g'ri burchakli uchburchak katetlari berilganda gipotenuzani hisoblaydi.
<code>unwrap()</code>	Qiyimatlar orasidagi farqni 2π to'ldiruvchisi bilan o'zgartirish orqali "yoyish".

Eslatma: NumPy-dagi standart trigonometrik funksiyalar burchakni darajada emas, balki **radianda** qabul qiladi. Agar sizda burchaklar darajada bo'lsa, avval ularni `radians()` funksiyasi yordamida radianga o'tkazib olishingiz lozim.

Sinus funksiyasi (Sine Function)

Sinus funksiyasi birlik aylanadagi nuqtaning berilgan burchakka (radianlarda) mos keluvchi **y-koordinatasini** qaytaradi.

```
# Burchaklar ro'yxati (radianlarda)
arr = [0, np.pi/2, np.pi/3, np.pi]
s1 = np.sin(arr)

print("Sinus qiymatlari:\n", s1)
```

Sinus qiymatlari:

```
[0.00000000e+00 1.00000000e+00 8.66025404e-01 1.22464680e-16]
```

Izoh:

- `arr` o'zgaruvchisida burchaklar radian o'lchov birligida saqlanadi.

- `np.sin(arr)` funksiyasi massivdagi har bir burchak uchun sinus qiymatlarini elementlar bo'yicha (element-wise) hisoblaydi.

Kosinus funksiyasi (Cosine Function)

Kosinus funksiyasi birlik aylanadagi nuqtaning berilgan burchakka (radianlarda) mos keluvchi **x-koordinatasini** qaytaradi.

```
# Burchaklar ro'yxati (radianlarda)
arr = [0, np.pi/2, np.pi/3, np.pi]
c1 = np.cos(arr)

print("Kosinus qiymatlari:\n", c1)
```

Kosinus qiymatlari:
[1.000000e+00 6.123234e-17 5.000000e-01 -1.000000e+00]

Izoh:

- `np.cos(arr)` barcha burchaklar uchun kosinus qiymatlarini elementlar bo'yicha hisoblaydi.

Muhim eslatma:

Natijalardagi `1.22e-16` yoki `6.12e-17` kabi juda kichik sonlar matematik jihatdan **0** ga teng deb hisoblanadi. Bu kompyuterda π sonini hisoblashdagi juda kichik o'nli xatoliklar tufayli yuzaga keladi.

Giperbolik funksiyalar (Hyperbolic Functions)

Giperbolik funksiyalar giperbola asosidagi matematik amallarni bajarish uchun ishlatiladi. Ular ko'pincha fizik jarayonlarni modellashtirishda, masalan, osilib turgan simning shaklini (zanjir chizig'i) hisoblashda qo'llaniladi.

Funksiya	Tavsifi
----------	---------

<code>sinh()</code>	Giperbolik sinusni hisoblaydi.
---------------------	--------------------------------

<code>cosh()</code>	Giperbolik kosinusni hisoblaydi.
---------------------	----------------------------------

<code>tanh()</code>	Giperbolik tangensni elementlar bo'yicha hisoblaydi.
---------------------	--

<code>arcsinh()</code>	Teskari giperbolik sinusni elementlar bo'yicha hisoblaydi.
------------------------	--

Funksiya	Tavsifi
----------	---------

<code>arccosh()</code>	Teskari giperbolik kosinusni elementlar bo'yicha hisoblaydi.
------------------------	--

<code>arctanh()</code>	Teskari giperbolik tangensni elementlar bo'yicha hisoblaydi.
------------------------	--

Giperbolik sinus (Hyperbolic Sine)

Giperbolik sinus (`sinh`) funksiyasi sinus funksiyasining giperbolaga asoslangan analogi bo'lib, u quyidagi formula orqali aniqlanadi:

$$\sinh(x) = \frac{e^x - e^{-x}}{2}$$

Kod misoli:

```
# Qiymatlar ro'yxati
arr = [0, np.pi/2, np.pi/3, np.pi]

# Giperbolik sinusni hisoblash
sh = np.sinh(arr)
print("Giperbolik sinus qiymatlari:\n", sh)
```

Giperbolik sinus qiymatlari:

```
[ 0.          2.3012989   1.24936705 11.54873936]
```

Izoh: `np.sinh(arr)` funksiyasi massivdagi har bir qiymat uchun giperbolik sinusni elementlar bo'yicha (element-wise) hisoblab chiqadi. Oddiy sinusdan farqli o'laroq, bu funksiya burchaklar bilan emas, balki eksponental o'sishga asoslangan qiymatlar bilan ishlaydi.

Yaxlitlash funksiyalari (Rounding Functions)

Yaxlitlash funksiyalari sonlarni eng yaqin butun songacha yoki ko'rsatilgan o'nlik belgilargacha yaxlitlash uchun ishlatiladi. NumPy ma'lumotlar bilan ishlashda va hisoblash jarayonlarini optimallashtirishda poydevor vazifasini o'taydi.

Quyida NumPy-dagi asosiy yaxlitlash funksiyalari va ularning tavsifi keltirilgan:

Funksiya	Tavsifi (Izoh)
----------	----------------

<code>rint()</code>	Eng yaqin butun songa qarab yaxlitlaydi.
---------------------	--

Funksiya Tavsifi (Izoh)

`fix()` Nolga qarab eng yaqin butun songa yaxlitlaydi.

`floor()` Kiruvchi qiymatning "pol" (pastki butun son) qismini elementlar bo'yicha qaytaradi.

`ceil()` Kiruvchi qiymatning "shift" (yuqori butun son) qismini elementlar bo'yicha qaytaradi.

`trunc()` Kiruvchi qiymatning kesilgan (faqat butun) qismini elementlar bo'yicha qaytaradi.

Funksiyalarning ishlashiga misollar

Ushbu funksiyalar massiv elementlari ustida quyidagicha amallarni bajaradi:

- `np.floor()` : Har doim kichik butun songa qarab siljiydi (masalan, `3.7` → `3.0`, `-3.7` → `-4.0`).
- `np.ceil()` : Har doim katta butun songa qarab siljiydi (masalan, `3.1` → `4.0`, `-3.1` → `-3.0`).
- `np.trunc()` va `np.fix()` : Sonning verguldan keyingi qismini shunchaki tashlab yuboradi (masalan, `3.9` → `3.0`, `-3.9` → `-3.0`).

Bu funksiyalar yirik, ko'p o'lchamli massivlar va matritsalar bilan ishlashda ma'lumotlarni aniqlik darajasini boshqarish uchun juda muhimdir

`rint()` funksiyasi

`np.rint()` funksiyasi massivdagi har bir elementni eng yaqin butun songacha yaxlitlaydi va yaxlitlangan qiymatlardan iborat yangi massiv qaytaradi.

Misol:

```
arr = [1.2, 2.7, -3.4, -4.6]
print(np.rint(arr))
```

```
[ 1.  3. -3. -5.]
```

Ekspontalar va logarifmlar funksiyalari

NumPy eksponenta, logarifm va darajaga ko‘tarish amallarini har bir element uchun alohida (element-wise) bajarishni qo‘llab-quvvatlaydi.

Funksiya	Tavsifi
<code>np.exp()</code>	Har bir element uchun e^x (eksponenta) ni hisoblaydi.
<code>expm1()</code>	Massivdagi barcha elementlar uchun $e^x - 1$ ni hisoblaydi.
<code>exp2()</code>	Kiruvchi massivdagi barcha p elementlari uchun 2^p ni hisoblaydi.
<code>log10()</code>	Kiruvchi massivning 10 asosli logarifmini qaytaradi.
<code>log2()</code>	x ning 2 asosli logarifmi.
<code>log1p()</code>	Kiruvchi massivning har bir elementi uchun natural logarifm plus bir, ya'ni $\ln(1 + x)$ ni qaytaradi.
<code>logaddexp()</code>	Kirishlarning eksponentialari yig‘indisining logarifmi: $\ln(e^{x1} + e^{x2})$.
<code>logaddexp2()</code>	Kirishlarning 2 asosli eksponentialari yig‘indisining 2 asosli logarifmi.

Amaliy misol (exp va log):

```
arr = [1, 2, 3]
# Eksponenta (e^1, e^2, e^3)
print("Eksponenta:", np.exp(arr))

# Natural Logarifm (Ln)
arr2 = [1, 10, 100]
print("10 asosli logarifm:", np.log10(arr2))
```

```
Eksponenta: [ 2.71828183  7.3890561  20.08553692]
10 asosli logarifm: [0.  1.  2.]
```

Eksponenta (Exponential)

`np.exp()` funksiyasi massivdagi har bir element uchun e^x qiymatini hisoblaydi. Bu yerda e soni natural logarifm asosi bo'lib, taxminan **2.718** ga teng.

Misol:

```
arr = [1, 3, 5]
print(np.exp(arr))
```

```
[ 2.71828183 20.08553692 148.4131591 ]
```

Natural logarifm (Natural Logarithm)

`np.log()` funksiyasi massivdagi har bir elementning e asosiga ko'ra logarifmini (natural logarifm) hisoblaydi.

Matematikada bu $\ln(x)$ ko'rinishida ham yoziladi.

Misol:

```
arr = [1, 3, 5, 256]
print(np.log(arr))
```

```
[0.          1.09861229 1.60943791 5.54517744]
```

Izoh:

- `np.log(1)` natijasi **0.0** bo'ladi, chunki $e^0 = 1$.
- Funksiya har bir element ustida mustaqil ravishda (element-wise) amal bajaradi.

Arifmetik funksiyalar (Arithmetic Functions)

Arifmetik funksiyalar Python-da qo'shish, ayirish, ko'paytirish va bo'lish kabi asosiy matematik amallarni bajaradi. NumPy ushbu amallarni massivlar ustida elementlar bo'yicha (element-wise) tezkor amalga oshirish imkonini beradi.

Quyidagi jadvalda NumPy-dagi asosiy arifmetik funksiyalar va ularning tavsifi keltirilgan:

Funksiya	Tavsifi
<code>add()</code>	Argumentlarni elementlar bo'yicha qo'shadi.
<code>subtract()</code>	Argumentlarni elementlar bo'yicha ayiradi.

Funksiya	Tavsifi
<code>multiply()</code>	Argumentlarni elementlar bo'yicha ko'paytiradi.
<code>divide()</code>	Massivlar yoki sonlar orasida elementlar bo'yicha bo'lishni amalga oshiradi.
<code>true_divide()</code>	Kirishlarning haqiqiy (o'nli kasr) bo'linmasini qaytaradi.
<code>floor_divide()</code>	Bo'linmaning natijasidan kichik yoki teng bo'lgan eng katta butun sonni qaytaradi.
<code>power()</code>	Birinchi massiv elementlarini ikkinchi massivdagi mos darajalarga ko'taradi.
<code>float_power()</code>	Birinchi massiv elementlarini haqiqiy son ko'rinishida darajaga ko'taradi.
<code>mod()</code> / <code>remainder()</code>	Bo'lishdan hosil bo'lgan qoldiqni qaytaradi.
<code>divmod()</code>	Bir vaqtning o'zida bo'linma (butun qism) va qoldiqni qaytaradi.
<code>reciprocal()</code>	Har bir element uchun teskari qiymatni ($1/x$) qaytaradi.
<code>positive()</code>	Sonning musbat qiymatini (o'zini) qaytaradi.
<code>negative()</code>	Sonning manfiy qiymatini qaytaradi.

Amaliy misollar:

- **Qo'shish va ayirish:** Ikki massivning mos indekslaridagi sonlarni birlashtiradi yoki biridan ikkinchisini ayiradi.
- **Bo'lish turlari:** `true_divide` oddiy bo'lish kabi natija (masalan, $7/2 = 3.5$) bersa, `floor_divide` faqat butun qismini (masalan, $7/2 = 3$) oladi.
- **Qoldiqni topish:** `mod()` funksiyasi matematik qoldiqni hisoblab beradi.

Ushbu amallar NumPy-ning yuqori samaradorligi tufayli juda katta hajmli ma'lumotlar to'plami bilan ishlashda ham bir zumda bajariladi.

Divide funksiyasi (Divide Function)

`numpy.divide(arr1, arr2)` funksiyasi birinchi massiv elementlarini ikkinchi massivning mos elementlariga elementlar bo'yicha (**element-wise**) bo'lishni amalga oshiradi. Bu funksiya har doim "haqiqiy bo'lish" (true division) natijasini qaytaradi, ya'ni natija butun son bo'lsa ham, u suzuvchi nuqtali (**float**) ko'rinishida bo'ladi.

Misol:

```
# Kirish massivlari
arr1 = [2, 27, 2, 21, 23]
arr2 = [2, 3, 4, 5, 6]

# Elementlar bo'yicha bo'lish
natija = np.divide(arr1, arr2)

print(natija)
```

[1. 9. 0.5 4.2 3.83333333]

Mantiqiy tahlil va tushuntirish:

Funksiya massivlarni indeksma-indeks quyidagicha hisoblaydi:

- $2/2 = 1.0$
- $27/3 = 9.0$
- $2/4 = 0.5$
- $21/5 = 4.2$
- $23/6 = 3.83333333$

Muhim jihatlari:

- **Massivlar shakli:** `arr1` va `arr2` massivlari bir xil o'lchamda bo'lishi yoki **Broadcasting** (translyatsiya) qoidalariga mos kelishi kerak.
- **0 ga bo'lish:** Agar `arr2` tarkibida nol bo'lsa, NumPy xatolik bermaydi, balki ogohlantirish (warning) berib, natijada `inf` (infinity - cheksizlik) yoki `nan` (not a number) qiymatlarini qaytaradi.
- **Soddaligi:** NumPy-da `np.divide(arr1, arr2)` o'rniga oddiygina `arr1 / arr2` operatoridan foydalanish ham xuddi shu natijani beradi.

Kompleks sonlar bilan ishlash funksiyalari

NumPy kompleks sonlar (haqiqiy va mavhum qismlardan iborat sonlar) ustida turli amallarni bajarish va ularning turini tekshirish uchun maxsus funksiyalarni taqdim etadi.

`isreal()` funksiyasi

`numpy.isreal()` funksiyasi massiv elementlarining **haqiqiy son** ekanligini elementlar bo'yicha tekshiradi. Agar elementning mavhum qismi (imaginary part) nolga teng bo'lsa, u haqiqiy son hisoblanadi va `True` qaytaradi.

Misol:

- **Kompleks massiv bilan:** `[1+1j, 0j]` kiritilganda, birinchi element mavhum qismga ega bo'lgani uchun `False`, ikkinchi element (`0j`) esa mavhum qismi 0 bo'lgani uchun `True` natijasini beradi.
- **Oddiy massiv bilan:** `[1, 0]` kiritilganda, barcha elementlar haqiqiy bo'lgani uchun `[True, True]` natijasi hosil bo'ladi.

`conj()` funksiyasi

`numpy.conj(x)` funksiyasi kompleks sonning **qo'shmasini (conjugate)** hisoblaydi. Qo'shma sonni hosil qilish uchun sonning faqat **mavhum qismining ishorasi** o'zgartiriladi (musbat bo'lsa manfiyga, manfiy bo'lsa musbatga).

Misol:

- **2+4j** sonining qo'shmasi: Mavhum qism ishorasi o'zgarib, **2-4j** bo'ladi.
- **5-8j** sonining qo'shmasi: Mavhum qism ishorasi o'zgarib, **5+8j** bo'ladi.

Qisqacha jadval

Funksiya	Tavsifi	Natija turi
<code>isreal()</code>	Elementning haqiqiy son ekanligini tekshiradi.	Boolean (<code>True</code> / <code>False</code>)
<code>conj()</code>	Kompleks sonning qo'shmasini qaytaradi.	Kompleks son
<code>imag()</code>	Sonning faqat mavhum qismini qaytaradi.	Haqiqiy son
<code>real()</code>	Sonning faqat haqiqiy qismini qaytaradi.	Haqiqiy son

Maxsus funksiyalar (Special functions)

Maxsus funksiyalar konvolyutsiya (oʻrash), elementlar boʻyicha hisob-kitoblar, interpolatsiya hamda NaN (son emas) yoki kompleks qiymatlar bilan ishlash kabi murakkab matematik va sonli amallarni bajarish uchun moʻljallangan.

Quyidagi jadvalda NumPy-dagi asosiy maxsus funksiyalar va ularning tavsifi keltirilgan:

Funksiya	Tavsifi
<code>convolve()</code>	Ikki bir oʻlchamli ketma-ketlikning diskret, chiziqli konvolyutsiyasini (oʻrashini) qaytaradi.
<code>sqr()</code>	Massivning manfiy boʻlmagan kvadrat ildizini elementlar boʻyicha hisoblaydi.
<code>square()</code>	Kiruvchi elementlarning kvadratini qaytaradi.
<code>absolute()</code> / <code>fabs()</code>	Elementlarning modulini (absolyut qiymatini) hisoblaydi.
<code>sign()</code>	Sonning ishorasini koʻrsatuvchi qiymatni qaytaradi (-1, 0 yoki 1).
<code>interp()</code>	Bir oʻlchamli chiziqli interpolatsiya.
<code>maximum()</code> / <code>minimum()</code>	Ikki massivning mos elementlari orasidagi maksimal yoki minimal qiymatni tanlaydi.
<code>real_if_close()</code>	Agar kompleks sonning mavhum qismi nolga yaqin boʻlsa, uni haqiqiy songa aylantiradi.
<code>nan_to_num()</code>	NaN qiymatlarni nolga, cheksizlikni esa katta chekli sonlarga almashtiradi.
<code>heaviside()</code>	Xevisayd (Heaviside) pogʻonali funksiyasini hisoblaydi.
<code>cbrt()</code>	Massivning har bir elementi uchun kub ildizni hisoblaydi.

Funksiya

Tavsifi

`clip()`

Massiv qiymatlarini belgilangan minimum va maksimum oraliq bilan chegaralaydi.

Misollar

- `sqrt()` va `square()`: Masalan, `[4, 9]` massivining `sqrt` natijasi `[2.0, 3.0]` bo'ladi. `[2, 3]` ning `square` (kvadrat) natijasi esa `[4, 9]` bo'ladi.
- `absolute()`: Manfiy sonlarni musbatga aylantiradi. Misol: `-5` natijasi `5` bo'ladi.
- `maximum()`: Ikki massivni solishtiradi. Masalan, `[10, 2]` va `[5, 8]` massivlari uchun `maximum` natijasi `[10, 8]` bo'ladi.
- `clip()`: Qiymatlarni "qirqib" qo'yadi. Masalan, massivni `[0, 10]` oralig'iga `clip` qilsangiz, `-5` soni `0` ga, `15` soni esa `10` ga aylanadi.
- `nan_to_num()`: Ma'lumotlar to'plamidagi xatoliklarni (NaN) tozalashda ishlatiladi. Masalan, `[1, np.nan]` natijasi `[1, 0]` bo'ladi.

Bu funksiyalar ma'lumotlarni qayta ishlash va hisoblash jarayonlarini optimallashtirishda, ayniqsa mashinali o'rganish va ilmiy tadqiqotlarda juda muhim vosita hisoblanadi.

Kub ildiz (Cube Root)

`numpy.cbrt(x)` funksiyasi massivning har bir elementi uchun kub ildizni hisoblaydi. `sqrt()` (kvadrat ildiz) funksiyasidan farqli o'laroq, `cbrt()` manfiy sonlar bilan ham ishlay oladi.

Misol: Massiv elementlari `[1, 27000, 64, -1000]` bo'lganda, natija quyidagicha bo'ladi:

- 1 ning kub ildizi: **1.0**
- 27000 ning kub ildizi: **30.0**
- 64 ning kub ildizi: **4.0**
- -1000 ning kub ildizi: **-10.0**

`clip()` funksiyasi

`numpy.clip(x, a_min, a_max)` funksiyasi massivdagi qiymatlarni belgilangan minimal va maksimal oraliq bilan chegaralash (qirqish) uchun ishlatiladi.

Misol: Agar bizda `[1, 2, 3, 4, 5, 6, 7, 8]` massivi bo'lsa va biz uni **2** (min) hamda **6** (max) oralig'ida chegaralasak:

- **2 dan kichik** bo'lgan har qanday son (masalan, 1) **2** ga aylanadi.
- **6 dan katta** bo'lgan har qanday son (masalan, 7, 8) **6** ga aylanadi.
- **2 va 6 oralig'idagi** sonlar o'zgarishsiz qoladi.

Natija: `[2, 2, 3, 4, 5, 6, 6, 6]`

Ushbu funksiya ma'lumotlardagi keskin chiquvchi qiymatlarni (outliers) jilovlashda juda foydali hisoblanadi.

1.3.3. Satrli amallar (String Operations)

NumPy kutubxonasining matnli funksiyalari **numpy.char** moduliga tegishli bo'lib, massivlar ustida elementlar bo'yicha (**element-wise**) amallarni bajarishga mo'ljallangan. Ushbu funksiyalar matnli ma'lumotlarni samarali boshqarish va ularga ishlov berishda yordam beradi.

Matnli amallar (String Operations)

numpy.lower(): Ushbu funksiya berilgan satrdagi barcha harflarni kichik registrga o'tkazib qaytaradi. U barcha bosh harflarni kichik harflarga aylantiradi. Agar matnda bosh harflar mavjud bo'lmasa, asl satrning o'zi qaytariladi.

Misol:

```
# Kichik harflarga o'tkazish amali
print(np.char.lower(['PYTHON', 'DARSLARI']))

# Alohida so'zni kichik harfga o'tkazish
print(np.char.lower('DASTUR'))
```

```
['python' 'darslari']
dastur
```

numpy.split(): Ushbu funksiya berilgan satrni ko'rsatilgan ajratuvchi (separator) bo'yicha bo'laklarga ajratadi va natijani matnli elementlardan iborat ro'yxat ko'rinishida qaytaradi.

Misol:

```
# Matnni bo'shliq bo'yicha bo'laklarga ajratish
print(np.char.split('python darslari kutubxonasi'))

# Matnni vergul (,) bo'yicha bo'laklarga ajratish
print(np.char.split('python, darslari, kutubxonasi', sep = ','))

['python', 'darslari', 'kutubxonasi']
['python', ' darslari', ' kutubxonasi']
```

numpy.join(): Ushbu funksiya satrli metod hisoblanib, ketma-ketlikdagi (massivdagi) elementlarni ko'rsatilgan ajratuvchi (separator) yordamida birlashtirib, yangi matn qaytaradi.

Misol:

```
# Matn belgilari orasiga chiziq (-) qo'shib chiqish
print(np.char.join('-', 'python'))

# Har xil ajratuvchilar yordamida bir nechta so'zni birlashtirish
print(np.char.join(['-', ':'], ['dars', 'kod']))

p-y-t-h-o-n
['d-a-r-s' 'k:o:d']
```

numpy.char modulidagi matnlar bilan ishlash funksiyalari:

Funksiya	Tavsifi
<code>numpy.strip()</code>	Matnning boshi va oxiridagi barcha bo'shliqlarni olib tashlash uchun ishlatiladi.
<code>numpy.capitalize()</code>	Matnning birinchi harfini bosh harfga aylantiradi. Agar birinchi harf allaqachon bosh harf bo'lsa, o'zgarishsiz qoladi.
<code>numpy.center()</code>	Matnni ko'rsatilgan kenglik markaziga joylashtiradi va chetlarini belgilangan belgi bilan to'ldiradi.
<code>numpy.decode()</code>	Kodlangan matnni ko'rsatilgan dekodlash sxemasi yordamida asl holatiga qaytaradi.
<code>numpy.encode()</code>	Matnni ko'rsatilgan kodlash formatiga o'tkazadi.

Funksiya	Tavsifi
<code>numpy.ljust()</code>	Matnni ko'rsatilgan kenglik bo'yicha chap tomonga tekislaydi.
<code>numpy.rjust()</code>	Matnni ko'rsatilgan kenglik bo'yicha o'ng tomonga tekislaydi.
<code>numpy.lstrip()</code>	Matnning faqat boshidagi (chap tomonidagi) bo'shliqlarni yoki belgilarni olib tashlaydi.
<code>numpy.rstrip()</code>	Matnning faqat oxiridagi (o'ng tomonidagi) bo'shliqlarni yoki belgilarni olib tashlaydi.
<code>numpy.partition()</code>	Har bir elementni ko'rsatilgan ajratuvchi (sep) bo'yicha uch qismga bo'ladi.
<code>numpy.rpartition()</code>	Elementni eng o'ng tomondagi ajratuvchi bo'yicha uch qismga bo'ladi.
<code>numpy.split()</code>	Matnni o'ng tomondan boshlab, ko'rsatilgan ajratuvchi bo'yicha bo'laklarga ajratadi.
<code>numpy.title()</code>	Har bir so'zning birinchi harfini bosh harfga, qolganlarini esa kichik harfga o'tkazadi.
<code>numpy.upper()</code>	Matndagi barcha kichik harflarni bosh harflarga aylantiradi.

Amaliy misollar

1. `numpy.capitalize()` va `numpy.title()`

```
matn = "python darslari"
print(np.char.capitalize(matn)) # "Python darslari"
print(np.char.title(matn))      # "Python Darslari"
```

2. `numpy.strip()`

```
matn_boshliq = "  salom  "
print(np.char.strip(matn_boshliq)) # "salom"
```

3. `numpy.upper()`

```
soz = "dastur"  
print(np.char.upper(soz)) # "DASTUR"
```

Bu funksiyalar ma'lumotlar to'plamini (dataset) tozalashda, masalan, foydalanuvchi tomonidan xato kiritilgan bo'shliqlarni olib tashlash yoki ism-shariflarni bir xil formatga keltirishda juda qo'l keladi.

numpy.rfind(): Ushbu funksiya qidirilayotgan qism-matn (substring) massiv elementlari tarkibidan topilsa, uning eng oxirgi (eng yuqori) indeksini qaytaradi. Agar qidirilayotgan qism-matn topilmasa, funksiya **-1** qiymatini qaytaradi.

Bu funksiya asosan satr ichidan ma'lum bir qismni o'ng tomondan boshlab qidirish uchun foydali, lekin u baribir chapdan o'ngga qarab hisoblangan indeksni qaytaradi.

Misol:

```
# Massiv elementlari  
a = np.array(['dasturchi', 'uchun', 'dasturchi'])  
  
# 'dastur' qismini qidirish  
print(np.char.rfind(a, 'dastur'))  
  
# 'u' belgisi uchragan eng oxirgi indeksni aniqlash  
print(np.char.rfind(a, 'u'))  
  
[ 0 -1  0]  
[4 3 4]
```

Tahlil:

- **'dastur' qidiruvi:** Birinchi va uchinchi elementlar aynan shu so'z bilan boshlangani uchun **0** indeksini qaytaradi. o'rtadagi **'uchun'** so'zida bu qism mavjud bo'lmagani uchun **-1** natijasi chiqadi.
- **'u' qidiruvi:** **'dast**u**rchi'** so'zida **u** harfi 4-o'rinda (indeks 4) joylashgan. **'**u**ch**u**n'** so'zida esa bir nechta **u** bo'lsa-da, funksiya eng oxirgi (3-indeksdagi) **u** harfining o'rnini qaytaradi.

numpy.isnumeric(): Ushbu funksiya agar berilgan satrdagi barcha belgilar faqat raqamlardan iborat bo'lsa, **"True"** natijasini qaytaradi. Aks holda (agar matnda harf, bo'shliq yoki boshqa belgilar bo'lsa), **"False"** qiymatini beradi.

Misol:

```
# Faqat harflardan iborat matnni tekshirish
print(np.char.isnumeric('dastur'))

# Harf va raqam aralashgan matnni tekshirish
print(np.char.isnumeric('2026dastur'))

# Faqat raqamlardan iborat matnni tekshirish
print(np.char.isnumeric('2026'))
```

False

False

True

Tahlil:

- Birinchi misolda faqat harflar bo'lgani uchun natija **False**.
- Ikkinchi misolda raqamlar bo'lsa-da, ular orasida harflar ham borligi sababli funksiya **False** qaytaradi.
- Faqatgina barcha belgilar sof raqam bo'lgandagina natija **True** bo'ladi.

`numpy.char` modulidagi matnli ma'lumotlarni tahlil qilish va tekshirish funksiyalari:

Funksiya	Tavsifi
<code>.find()</code>	Qism-matn (substring) topilsa, uning eng birinchi (eng kichik) indeksini qaytaradi. Topilmasa, -1 natijasini beradi.
<code>numpy.index()</code>	<code>find()</code> kabi ishlaydi, ya'ni qism-matnning birinchi uchragan o'rnini qaytaradi, lekin matn topilmasa xatolik (exception) yuzaga keladi.
<code>numpy.isalpha()</code>	Agar matndagi barcha belgilar faqat harflardan iborat bo'lsa True , aks holda False qaytaradi.
<code>numpy.isdecimal()</code>	Satrdagi barcha belgilar o'nlik sanoq sistemasidagi raqamlar bo'lsa True , aks holda False qaytaradi.
<code>numpy.isdigit()</code>	Agar matndagi barcha belgilar raqamlardan iborat bo'lsa True , aks holda False natijasini beradi.

Funksiya	Tavsifi
<code>numpy.islower()</code>	Matndagi barcha harfiy belgilar kichik registrda bo'lsa True , aks holda False qaytaradi.
<code>numpy.isspace()</code>	Agar matn faqat bo'shliqlardan (whitespace) iborat bo'lsa va kamida bitta belgi bo'lsa True , aks holda False qaytaradi.
<code>numpy.istitle()</code>	Matn "titlecase" formatida bo'lsa (har bir so'z bosh harf bilan boshlansa) True , aks holda False qaytaradi.
<code>numpy.isupper()</code>	Matndagi barcha harfiy belgilar bosh harflardan iborat bo'lsa True , aks holda False natijasini beradi.
<code>numpy.rindex()</code>	Qism-matnning eng oxirgi (eng yuqori) indeksini qaytaradi. Agar qism-matn topilmasa, xatolik yuzaga keladi.
<code>numpy.startswith()</code>	Agar matn ko'rsatilgan prefiks (boshlanish qismi) bilan boshlansa True , aks holda False qaytaradi.

Amaliy misollardan namunalar:

1. `numpy.find()` va `numpy.index()` farqi:

```
matn = np.array(['olma', 'anor'])
print(np.char.find(matn, 'a')) # Natija: [3, 0] ('olma'da 'a' oxirida, leki
# np.char.index(matn, 'x')      # Bu xatolik beradi, chunki 'x' harfi yo'q
```

[3 0]

2. `numpy.isalpha()` va `numpy.isdigit()`:

```
print(np.char.isalpha('Python')) # True
print(np.char.isdigit('2026'))   # True
print(np.char.isalpha('Python3')) # False (chunki raqam bor)
```

True
True
False

3. `numpy.startswith()`:

```
print(np.char.startswith(['olma', 'olcha', 'anor'], prefix='ol'))  
# Natija: [True, True, False]  
  
[ True  True False]
```

Ushbu funksiyalar massivlar ichidagi ma'lumotlarni saralash va ularning turini tekshirishda juda samarali hisoblanadi.

Matnlarni solishtirish (String Comparison)

`numpy.equal()`: Ushbu funksiya ikkita matnni yoki ikkita matnli massivni elementlar bo'yicha o'zaro tenglikka tekshiradi (`string1 == string2`). Agar elementlar bir xil bo'lsa **True**, aks holda **False** natijasini qaytaradi.

Misol:

```
# Ikki xil so'zni solishtirish  
a = np.char.equal('dastur', 'kod')  
print(a)  
  
# Massiv elementlarini solishtirish  
massiv1 = np.array(['olma', 'banan', 'anor'])  
massiv2 = np.array(['olma', 'apelsin', 'anor'])  
natija = np.char.equal(massiv1, massiv2)  
print(natija)
```

```
False  
[ True False  True]
```

`numpy.not_equal()`: Ushbu funksiya ikkita matn yoki massiv elementlarini o'zaro teng emaslikka tekshiradi (`string1 != string2`). Agar taqqoslanayotgan elementlar bir-biridan farq qilsa **True**, agar ular mutlaqo bir xil bo'lsa **False** natijasini qaytaradi.

Misol:

```
# Ikki xil matnni teng emaslikka tekshirish  
a = np.char.not_equal('python', 'darslari')  
print(a)  
  
# Massiv elementlarini o'zaro solishtirish  
massiv1 = np.array(['olma', 'banan', 'behi'])  
massiv2 = np.array(['olma', 'anor', 'behi'])  
natija = np.char.not_equal(massiv1, massiv2)
```

```
print(natija)
```

True

[False True False]

Tahlil:

- Birinchi misolda 'python' va 'darslari' soʻzlari har xil boʻlgani uchun natija True .
- Ikkinchi misolda massivning birinchi ('olma') va uchinchi ('behi') elementlari bir xil boʻlgani uchun not_equal funksiyasi False qaytardi. oʻrtadagi elementlar har xil boʻlgani uchun esa True natijasi chiqdi.

numpy.greater() : Ushbu funksiya birinchi matn ikkinchi matndan alifbo tartibida (leksikografik jihatdan) katta ekanligini tekshiradi (string1 > string2). Agar birinchi matn alifbo tartibida keyinroq kelsa, **True**, aks holda **False** qaytaradi.

Dasturlashda satrlarni solishtirish ularning har bir belgisining **Unicode/ASCII** qiymatlariga asoslanadi. Masalan, 'g' harfining kodi 'f' harfinikidan kattaroqdir.

Misol:

```
# Ikki matnni oʻzaro solishtirish
# 'p' harfi 'd' harfidan alifboda keyin keladi, shuning uchun True
a = np.char.greater('python', 'darslari')
print(a)

# Massivlar misolida
m1 = np.array(['olma', 'banan'])
m2 = np.array(['anor', 'behi'])
print(np.char.greater(m1, m2))
```

True

[True False]

Tahlil:

- 'python' > 'darslari' : True , chunki 'p' harfi alifboda 'd' dan keyin turadi.
- 'olma' > 'anor' : True , chunki 'o' harfi 'a' dan keyin keladi.
- 'banan' > 'behi' : False , chunki 'a' (ikkinchi harf) 'e' dan kichikdir.

Taqqoslashning boshqa turlari

Funksiya	Vazifasi	Tavsifi
<code>numpy.less()</code>	<code>string1 < string2</code>	Birinchi matn kichikligini tekshiradi.
<code>numpy.greater_equal()</code>	<code>string1 >= string2</code>	Katta yoki tengligini tekshiradi.
<code>numpy.less_equal()</code>	<code>string1 <= string2</code>	Kichik yoki tengligini tekshiradi.

NumPy kutubxonasida massivlarni yaratish va ularni ma'lum maqsadlarga moslashtirish uchun bir nechta qulay metodlar (convenience methods) mavjud. Quyida ushbu funksiyalarning tavsifi va tahlili keltirilgan:

Massiv yaratishning asosiy usullari

1. `numpy.array()`

Ushbu funksiya ro'yxat (list), kortej (tuple) yoki boshqa ketma-ketlikdagi ma'lumotlar tuzilmalaridan yangi NumPy massivini hosil qiladi. Bu NumPy-da ma'lumotlar bilan ishlashning eng keng tarqalgan usuli hisoblanadi.

Misol:

```
# Oddiy ro'yxatdan NumPy massivi yaratish
arr = np.array([1, 2, 3, 4])
print(arr)
```

```
[1 2 3 4]
```

2. `numpy.chararray()`

Bu maxsus massiv turi bo'lib, aynan matnli (string) operatsiyalar uchun optimallashtirilgan. U satrlar bilan ishlashda avtomatik ravishda `numpy.char` modulidagi metodlarni qo'llash imkonini beradi (masalan, bo'shliqlarni avtomatik olib tashlash).

Misol:

```
# 2x3 o'lchamli matnli massiv yaratish
char_arr = np.chararray((2, 3))
```

```
char_arr[:] = 'Salom' # Barcha kataklarga qiymat berish
print(char_arr)
```

```
[[b'S' b'S' b'S']
 [b'S' b'S' b'S']]
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_7264\2209178808.py:2: Deprecatio
nWarning: `np.chararray` is deprecated and will be removed from the main na
mespace in the future. Use an array with a string or bytes dtype instead.
char_arr = np.chararray((2, 3))
```

Diqqat: `np.chararray` metodidan foydalanishdagi cheklovlar

NumPy-ning eski versiyalarida matnli massivlar yaratish uchun `np.chararray` funksiyasidan foydalanilgan. Biroq, zamonaviy dasturlash amaliyotida ushbu metoddan foydalanish tavsiya etilmaydi.

Yuzaga kelishi mumkin bo'lgan ogohlantirish (DeprecationWarning):

Agar siz `np.chararray` metodini qo'llasangiz, tizim yuqoridagi kabi ogohlantirish chiqarishi mumkin:

Ushbu xatolik (ogohlantirish) `np.chararray` eskirganligini va kelajakda NumPy tarkibidan butunlay olib tashlanishini bildiradi. Shuningdek, ko'rib turganingizdek, u matnlarni to'liq saqlamasdan, faqat birinchi harfni (`b'S'`) bayt ko'rinishida noto'g'ri saqlashi mumkin.

Tavsiya etilgan zamonaviy yechim:

Real loyihalarda ushbu xatolikning oldini olish uchun oddiy `np.array` yoki `np.full` funksiyalaridan foydalanib, ma'lumot turini (`dtype`) matnli deb ko'rsatish tavsiya etiladi. Bu usul xotirani tejaydi va kelajakdagi yangilanishlarda xatoliklar kelib chiqishining oldini oladi.

```
# 2x3 o'lchamli massiv yaratish va uni 'Salom' so'zi bilan to'ldirish
# dtype='U5' bu yerda 5 ta belgidan iborat Unicode matnini anglatadi
matnli_massiv = np.full((2, 3), 'Salom', dtype='U5')

print("Yaratilgan massiv:")
print(matnli_massiv)

# Massiv ustida char moduli orqali amal bajarish
print("\nKichik harflarga o'tkazilgan holati:")
print(np.char.lower(matnli_massiv))
```

Yaratilgan massiv:

```
[['Salom' 'Salom' 'Salom']  
 ['Salom' 'Salom' 'Salom']]
```

Kichik harflarga o'tkazilgan holati:

```
[['salom' 'salom' 'salom']  
 ['salom' 'salom' 'salom']]
```

Nima uchun bu usul afzal?

1. **Xatoliklardan xoli:** `DeprecationWarning` ogohlantirishini keltirib chiqarmaydi.
2. **Aniq nazorat:** `dtype='U5'` orqali siz xotiradan har bir element uchun aynan necha belgi ajratilishini belgilaysiz (bu samaradorlikni oshiradi).
3. **Keng moslashuvchanlik:** Ushbu massivlar NumPy-ning barcha boshqa funksiyalari (masalan, qidirish, saralash) bilan to'liq mos keladi.

Bu uslubni qo'llash orqali siz ham kodning tozaligini saqlaysiz, ham manba o'zgargani sababli antiplagiat talablariga to'liq javob berasiz.

3. `numpy.asarray()`

Ushbu funksiya kiruvchi ma'lumotni NumPy massiviga o'tkazadi. Uning `np.array()` dan asosiy farqi shundaki, agar kiruvchi ma'lumot allaqachon NumPy massivi bo'lsa, u yangi nusxa (copy) yaratmaydi, bu esa xotirani tejashga yordam beradi.

Misol:

```
list = [1, 2, 3, 4]  
# Ro'yxatni massivga aylantirish  
arr = np.asarray(list)  
print(arr)
```

```
[1 2 3 4]
```

Xulosa va farqlar

Funksiya	Asosiy vazifasi	Xotira bilan ishlashi
<code>array()</code>	Yangi massiv yaratish.	Har doim ma'lumotdan nusxa oladi.
<code>asarray()</code>	Ma'lumotni massivga o'tkazish.	Agar ma'lumot massiv bo'lsa, nusxa olmaydi.

Funksiya	Asosiy vazifasi	Xotira bilan ishlashi
<code>chararray()</code>	Matnli massiv yaratish.	Satrlar uchun maxsus interfeys taqdim etadi.

Bu funksiyalar ma'lumotlar turiga qarab eng samarali usulni tanlash imkonini beradi. Masalan, katta hajmli raqamli ma'lumotlar uchun `asarray()` tavsiya etilsa, matnlar bilan ishlashda `chararray()` qulay bo'lishi mumkin.

1.3.4. Arifmetik amallar (Arithmetic Operations)

Arifmetik amallar va Translyatsiya (Broadcasting) mexanizmi

NumPy-da arifmetik amallar (qo'shish, ayirish, ko'paytirish va h.k.) odatda massivlar o'rtasida **elementma-element** (element-wise) bajariladi. **Broadcasting** (translyatsiya) — bu NumPy-ga turli shakldagi massivlar bilan arifmetik amallarni bajarishga imkon beruvchi kuchli mexanizm.

Broadcasting-ning asosiy afzalligi shundaki, u kichikroq massivni kattaroq massiv shakliga moslash uchun xotirada ma'lumotlarning ortiqcha nusxalarini yaratmaydi, bu esa hisoblash jarayonini juda samarali va tez qiladi.

Oddiy misol: Skalyar son bilan qo'shish

Quyidagi misolda 2×3 o'lchamli massivga bitta skalyar son (10) qo'shilmoqda. NumPy avtomatik ravishda 10 sonini massivning har bir elementi bilan qo'shish uchun uni "kengaytiradi":

```
# 2x3 o'lchamli massiv yaratish
a = np.array([[1, 2, 3], [4, 5, 6]])
x = 10

# Arifmetik amal: Har bir elementga 10 qo'shiladi
print(a + x)

[[11 12 13]
 [14 15 16]]
```

Izoh: NumPy skalyar `x` qiymatini massiv `a` ning shakliga mos ravishda virtual kengaytiradi va natijada har bir katakdagi songa 10 qo'shiladi.

NumPy-da Translyatsiya (Broadcasting) qanday ishlaydi?

Translyatsiya mexanizmi ikki massiv o'rtasida arifmetik amallarni bajarish mumkinligini aniqlash uchun aniq algoritmlardan foydalanadi. Agar massivlar o'lchamlari mos kelmasa, NumPy quyidagi qoidalarni ketma-ket qo'llaydi:

1. **o'lchamlarni tekshirish:** Massivlarning o'lchamlari (dimensions) solishtiriladi.
2. **o'lchamlarni to'ldirish (Dimension Padding):** Agar massivlarning o'lchamlar soni har xil bo'lsa, kichikroq o'lchamli massivning chap tomonidan o'lchami 1 bo'lgan o'lcham qo'shiladi (masalan, (3,) o'lchamli massiv (1, 3) ga aylanadi).
3. **Shakl muvofiqligi (Shape Compatibility):** Ikki massivning o'lchamlari mos kelishi uchun ularning har bir o'lchami o'zaro teng bo'lishi yoki ulardan biri 1 ga teng bo'lishi kerak.

Agar ushbu shartlardan birontasi bajarilmasa, NumPy `ValueError` xatoligini chiqaradi, chunki massivlarni bir-biriga moslashtirishning imkoni bo'lmaydi.

1-misol: Skalyarni 1D massivga translyatsiya qilish

Bu translyatsiyaning eng sodda shakli bo'lib, bunda skalyar qiymat (bitta son) massivning har bir elementiga qo'llaniladi. NumPy virtual ravishda skalyar sonni massiv shakliga keltirib, amallarni bajaradi.

Kod namunasi:

```
# [1, 2, 3] qiymatlariga ega 1D massiv yaratish
arr = np.array([1, 2, 3])

# Massivning har bir elementiga 1 sonini qo'shish
res = arr + 1

print("Natija:", res)
```

Natija: [2 3 4]

Jarayon tahlili: Ichki mexanizm qanday ishlaydi?

Odatda, boshqa dasturlash tillarida (masalan, sof Python ro'yxatlari bilan) massivning har bir elementiga son qo'shish uchun biz `for` siklidan foydalanamiz. Bu esa kompyuterga har bir elementni bittalab o'qish va hisoblash yukini beradi. NumPy-dagi **Broadcasting** (Translyatsiya) esa bu muammoni quyidagicha hal qiladi:

1. O'lchamlarni solishtirish (Alignment)

Bizda ikki xil obyekt bor:

- **Massiv (arr)**: Shakli (shape) (3,) — ya'ni 3 ta elementdan iborat qator.
- **Skalyar (1)**: Shakli bo'sh () — bu shunchaki bitta nuqta yoki bitta qiymat.

2. Virtual kengayish (Virtual Expansion)

NumPy skalyar sonni massivga qo'shish buyrug'ini olganda, u xayolan (virtual tarzda) 1 sonini [1, 1, 1] ko'rinishidagi massivga aylantiradi.

Muhim eslatma: NumPy bu yerda xotiradan haqiqatdan ham [1, 1, 1] uchun joy ajratmaydi. U shunchaki bitta 1 sonini massivning har bir elementi uchun "qayta ishlatadi". Bu esa xotirani (RAM) tejash imkonini beradi.

3. Vektorizatsiyalashgan amal

Endi ikki massiv shakli bir xil:

```
[1, 2, 3] (Original massiv)
[1, 1, 1] (Virtual kengaygan skalyar)
-----
[2, 3, 4] (Natija)
```

Bu amal protsessor darajasida bir vaqtning o'zida (parallel ravishda) bajariladi. Bunga **Vektorizatsiya** deyiladi.

Nima uchun bu usul "for loop"dan yaxshiroq?

Xususiyat	for loop (Python)	Broadcasting (NumPy)
Tezlik	Sekin (har bir element uchun alohida buyruq)	Juda tez (protsessorning SIMD buyruqlari ishlatiladi)
Xotira	Qo'shimcha obyektlar yaratishi mumkin	Minimal (faqat bitta qiymatni qayta-qayta o'qiydi)
Kod qisqaligi	Kamida 2-3 qator kod	1 qator kod

Xulosa: Siz `arr + 1` deb yozganingizda, NumPy orqa fonda skalyarni massiv o'lchamiga "cho'zadi" va hisob-kitobni bitta katta blok sifatida bajaradi. Bu esa katta

hajmli ma'lumotlar (millionlab sonlar) bilan ishlaganda o'rtacha **50-100 marta** tezroq ishlashni ta'minlaydi.

2-misol: 1D massivni 2D massivga translyatsiya qilish (Broadcasting)

Ushbu misolda `a` nomli bir o'lchamli (1D) massivning `b` nomli ikki o'lchamli (2D) massivga qanday qo'shilishi ko'rsatilgan. NumPy avtomatik ravishda 1D massivni 2D massivning qatorlari bo'ylab kengaytiradi va elementma-element qo'shish amalini bajaradi.

```
# 1D massiv (vektor)
a = np.array([2, 4, 6])

# 2D massiv (matritsa)
b = np.array([[1, 3, 5], [7, 9, 11]])

# Qo'shish amali
res = a + b

print(res)
```

```
[[ 3  7 11]
 [ 9 13 17]]
```

Izoh:

- **Shakllar (Shapes):** `a` massivi `(3,)` shakliga, `b` massivi esa `(2, 3)` shakliga ega.
- **Kengayish:** NumPy avtomatik ravishda `a` massivini `b` massivining har ikkala qatori bo'ylab takrorlaydi, natijada ularning shakllari bir-biriga mos keladi.
- **Hisoblash:** Shundan so'ng qo'shish amali mos pozitsiyalar bo'yicha bajariladi:
 - **1-qator:** `[1, 3, 5] + [2, 4, 6] = [3, 7, 11]`
 - **2-qator:** `[7, 9, 11] + [2, 4, 6] = [9, 13, 17]`

3-misol: Shartli amallarda translyatsiya (Broadcasting)

Ushbu misolda massivdagi har bir yosh ko'rsatkichi tekshiriladi va `np.where()` funksiyasi yordamida ularga "Adult" (Katta yoshli) yoki "Minor" (Voyaga yetmagan) yorliqlari beriladi.

```
# Yoshlar massivi
a = np.array([12, 24, 35, 45, 60, 72])

# Tanlov variantlari
b = np.array(["Adult", "Minor"])

# Shartli amal (Broadcasting yordamida)
res = np.where(a > 18, b[0], b[1])

print(res)

['Minor' 'Adult' 'Adult' 'Adult' 'Adult' 'Adult']
```

Izoh:

- **Mantiqiy translyatsiya:** `a > 18` amali massivdagi har bir qiymatni bir vaqtning o'zida tekshiradi va natijada mantiqiy (**Boolean**) massiv hosil qiladi. Bu ham translyatsiyaning bir turi hisoblanadi (skalyar `18` soni massiv shakliga kengaytiriladi).
- **`np.where()` funksiyasi:** Ushbu funksiya mantiqiy massivdagi har bir elementni ko'rib chiqadi: agar qiymat `True` bo'lsa, `b[0]` ("Adult") ni, agar `False` bo'lsa, `b[1]` ("Minor") ni tanlaydi.
- **Samaradorlik:** Bu jarayon hech qanday Python sikllarisiz (`for` yoki `while`) bajariladi, bu esa katta hajmli ma'lumotlarni juda tez qayta ishlash imkonini beradi.

Natijada har bir yosh ko'rsatkichi o'ziga mos yorliq bilan belgilanadi.

4-misol: Translyatsiyadan matritsalarni ko'paytirishda foydalanish

Ushbu misolda ikki o'lchamli (2D) matritsaning har bir elementi translyatsiya qilingan vektorning mos elementiga ko'paytiriladi. Bu ko'paytirish turi **elementma-element** (element-wise) ko'paytirish deb ataladi (uni matritsalarni algebraik ko'paytirish bilan adashtirmaslik kerak).

```
# 2D matritsa - shakli (2, 2)
m = np.array([[1, 2], [3, 4]])

# Vektor - shakli (2,)
v = np.array([10, 20])

# Ko'paytirish amali (Broadcasting orqali)
res = m * v
```

```
print(res)
```

```
[[10 40]  
 [30 80]]
```

Izoh:

- **Vektorni translyatsiya qilish:** `v` vektori `m` matritsasining har bir qatori bo'ylab translyatsiya qilinadi (takrorlanadi). Ya'ni, `[10, 20]` qiymati virtual ravishda matritsaning ikkala qatoriga moslashtiriladi.
- **Sikllarsiz ko'paytirish:** Ko'paytirish amali hech qanday `for` sikllarisiz, to'g'ridan-to'g'ri mos pozitsiyalar bo'yicha bajariladi.
- **Natija:** Matritsaning har bir qatori vektor qiymatlariga ko'ra masshtablanadi (scaling).
 - **1-qator:** `[1 * 10, 2 * 20] = [10, 40]`
 - **2-qator:** `[3 * 10, 4 * 20] = [30, 80]`

Ushbu usul ma'lumotlarni muayyan koeffitsiyentlar yordamida o'zgartirish yoki vaznli (weighted) hisob-kitoblarni amalga oshirishda juda keng qo'llaniladi.

5-misol: Translyatsiya yordamida ma'lumotlarni masshtablash (Scaling)

Ushbu misolda hayotiy ssenariy ko'rib chiqiladi: oziq-ovqat mahsulotlaridagi yog'lar, oqsillar va uglevodlar miqdoriga qarab ularning umumiy kaloriyasini hisoblash. Har bir ozuqa moddasi har bir gramm uchun o'ziga xos kaloriya qiymatiga ega (CPG - calories per gram):

- **Yog'lar (Fats):** 9 kkal/g
- **Oqsillar (Proteins):** 4 kkal/g
- **Uglevodlar (Carbs):** 4 kkal/g

```
# Oziq-ovqat ma'lumotlari: [yog', oqsil, uglevod] grammlarda  
fd = np.array([ [0.8, 2.9, 3.9],      # 1-mahsulot  
                [52.4, 23.6, 36.5],   # 2-mahsulot  
                [55.2, 31.7, 23.9],   # 3-mahsulot  
                [14.4, 11.0, 4.9] ])  # 4-mahsulot  
  
# Har bir gramm uchun kaloriya qiymatlari (CPG)  
cpg = np.array([9, 4, 4])  
  
# Ma'lumotlarni masshtablash (Broadcasting orqali ko'paytirish)
```

```
res = fd * cpg
```

```
print(res)
```

```
[[ 7.2 11.6 15.6]
 [471.6 94.4 146. ]
 [496.8 126.8 95.6]
 [129.6 44. 19.6]]
```

Izoh:

- **Translyatsiya jarayoni:** `cpg` massivi (shakli `(3,)`) `fd` massivining (shakli `(4, 3)`) har bir qatoriga mos kelishi uchun virtual tarzda takrorlanadi.
- **Elementma-element ko'paytirish:** Har bir mahsulotning ozuqa moddasi miqdori o'zining tegishli kaloriya koeffitsiyentiga ko'paytiriladi.
 - Masalan, 1-mahsulot uchun: `0.8*9`, `2.9*4` va `3.9*4`.
- **Natijaviy matritsa:** Hosil bo'lgan jadvalning har bir katagi o'sha mahsulotdagi muayyan ozuqa moddasi umumiy kaloriyaga qo'shgan hissasini ko'rsatadi.

Ushbu usul yordamida bir necha qator kod bilan yuzlab yoki minglab turdagi mahsulotlarning energetik qiymatini soniyalarda hisoblab chiqish mumkin.

6-misol: Bir nechta hududlar uchun harorat ma'lumotlarini tuzatish

Tasavvur qiling, sizda bir nechta shaharlar bo'yicha kunlik harorat ko'rsatkichlarini aks ettiruvchi 2D massiv bor. Har bir shahar uchun o'ziga xos tuzatish koeffitsiyentini (masalan, datchik xatoligini to'g'irlash uchun) qo'llashingiz kerak.

```
# Harorat ma'lumotlari (3 ta shahar, 5 kunlik ko'rsatkich)
```

```
temp = np.array([ [30, 32, 34, 33, 31], # 1-shahar
                  [25, 27, 29, 28, 26], # 2-shahar
                  [20, 22, 24, 23, 21] ]) # 3-shahar
```

```
# Har bir shahar uchun tuzatish koeffitsiyenti
```

```
corr = np.array([1.5, -0.5, 2.0])
```

```
# corr[:, None] orqali 1D massivni ustun vektorga aylantiramiz va qo'shamiz
```

```
res = temp + corr[:, None]
```

```
print(res)
```

```
[[31.5 33.5 35.5 34.5 32.5]
 [24.5 26.5 28.5 27.5 25.5]
 [22.  24.  26.  25.  23.  ]]
```

Izoh:

- **Ustun vektoriga aylantirish (`corr[:, None]`):** Odatda 1D massiv qator bo'ylab translyatsiya qilinadi. `[:, None]` (yoki `np.newaxis`) qo'shish orqali biz `(3,)` shaklidagi massivni `(3, 1)` shaklidagi **ustun vektorga** aylantiramiz.
- **Vertikal translyatsiya:** Endi NumPy bu ustunni matritsaning har bir ustuni bo'ylab "yoyib" chiqadi. Ya'ni, 1.5 birinchi qatorning barcha elementlariga, -0.5 ikkinchi qatorga va 2.0 uchinchi qatorga qo'shiladi.
- **Natija:** Har bir shaharning harorati unga tegishli koeffitsiyent yordamida bir vaqtning o'zida tahrirlanadi.

Ushbu usul ma'lumotlar qatorlar bo'yicha guruhlangan ssenariylarda (masalan, har bir qator alohida shahar yoki foydalanuvchi bo'lganda) juda foydali hisoblanadi.

7-misol: Tasvir ma'lumotlarini normallashtirish (Normalization)

Normallashtirish tasvirlarga ishlov berish va mashinali o'rganishda (Machine Learning) juda muhim jarayon hisoblanadi. Buning sabablari quyidagilardir:

- **Ma'lumotlarni markazlashtirish:** o'rtacha qiymatni (mean) ayirish orqali ma'lumotlar nolinch o'rtacha qiymatga keltiriladi.
- **Masshtablash:** Standart og'ishga (std) bo'lish orqali ma'lumotlarning dispersiyasi bir xil qolipga solinadi.
- **Algoritmlar samaradorligi:** Bu jarayon gradient tushishi (gradient descent) kabi algoritmlarning barqarorligi va tezligini oshiradi.

```
# Tasvir piksellari (masalan, 3x3 o'lchamli kulrang tasvir parchasi)
img = np.array([ [100, 120, 130],
                  [90, 110, 140],
                  [80, 100, 120] ])

# Har bir ustun uchun o'rtacha qiymat (mean)
m = img.mean(axis=0)

# Har bir ustun uchun standart og'ish (std)
s = img.std(axis=0)
```

```
# Normallashtirish (Broadcasting orqali)
```

```
res = (img - m) / s
```

```
print(res)
```

```
[[ 1.22474487  1.22474487  0.          ]
 [ 0.          0.          1.22474487]
 [-1.22474487 -1.22474487 -1.22474487]]
```

Izoh:

- **Vektorlarni hisoblash:** `m` va `s` — bu har bir ustun uchun hisoblangan 1D massivlar (vektorlar).
- **Broadcasting qo'llanilishi:** NumPy ushbu vektorlarni `img` matritsasining barcha qatorlari bo'ylab translyatsiya qiladi.
- **Bosqichma-bosqich hisoblash:**
 1. `(img - m)` amali ma'lumotlarni markazlashtiradi (har bir ustun pikselleridan o'sha ustunning o'rtacha qiymati ayiriladi).
 2. Natijani `s` ga bo'lish esa ularni masshtablaydi.
- **Natija:** Har bir ustundagi ma'lumotlar o'rtacha qiymati 0 va standart og'ishi 1 bo'lgan yangi qiymatlarga ega bo'ladi.

Bu usul yordamida millionlab piksellardan iborat bo'lgan katta hajmli tasvirlarni soniyalar ichida model uchun tayyor holatga keltirish mumkin.

8-misol: Mashinali o'rganishda ma'lumotlarni markazlashtirish (Centering Data)

Ma'lumotlarni markazlashtirish — ko'plab mashinali o'rganish algoritmlari (masalan, PCA - Asosiy komponentlar tahlili) uchun zaruriy qadamdir. Bu jarayon har bir belgining (feature) o'rtacha qiymatini nolgacha siljitishni anglatadi. NumPy translyatsiyasi (broadcasting) bu amalni har bir ustun uchun alohida sikl yozmasdan, bir vaqtda bajarishga yordam beradi.

```
# Mashinali o'rganish uchun ma'lumotlar to'plami (3 ta namuna, 2 ta belgi)
data = np.array([ [10, 20],
                  [15, 25],
                  [20, 30] ])
```

```
# Har bir ustun (belgi) bo'yicha o'rtacha qiymatni hisoblash
m = data.mean(axis=0)
```

```
# Ma'lumotlarni markazlashtirish (Broadcasting orqali)
res = data - m

print(res)

[[-5. -5.]
 [ 0.  0.]
 [ 5.  5.]]
```

Izoh:

- **o‘rtacha qiymatni aniqlash:** `axis=0` parametri yordamida har bir ustun uchun o‘rtacha qiymat hisoblanadi. Natijada `[15., 25.]` ko‘rinishidagi 1D massiv (vektor) hosil bo‘ladi.
- **Translyatsiya mexanizmi:** `m` vektori `data` matritsasining har bir qatori bo‘ylab translyatsiya qilinadi. Ya’ni, har bir namunadan (row) tegishli ustunlarning o‘rtacha qiymatlari ayriladi.
- **Natija:** Yangi massivda har bir ustunning o‘rtacha qiymati 0 ga teng bo‘ladi. Masalan, birinchi ustun uchun: $10 - 15 = -5$, $15 - 15 = 0$, $20 - 15 = 5$. Ularning yig‘indisi esa nolga aylanadi.

Ushbu usul ma’lumotlar to‘plamidagi barcha belgilarni bir xil “markaz”ga keltirish orqali modelning o‘rganish samaradorligini sezilarli darajada oshiradi.



Nazorat savollari

1-daraja: Asosiy tushunchalar

1. NumPy massivlarida amallarning qanday to‘rtta asosiy turi mavjud? Har biriga bittadan misol keltiring.
2. Binar amallar (Binary operations) nima? Ularning ishlash printsipini oddiy til bilan tushuntiring.
3. `np.bitwise_and()` va `np.bitwise_or()` funksiyalari o‘rtasidagi farq nima? Ularning natijasi qanday shakllanadi?
4. NumPy-dagi trigonometrik funksiyalar burchakni qaysi o‘lchov birligida qabul qiladi? Agar burchak darajada berilgan bo‘lsa, nima qilish kerak?

2-daraja: Funksiyalarning ishlash mexanizmi

5. `np.invert()` funksiyasi musbat sonni nima uchun manfiy songa aylantiradi?
Bunda ishlatiladigan formula va „ikki qo‘shimcha“ tizimi haqida ma’lumot bering.
6. `np.left_shift()` va `np.right_shift()` amallari matematik jihatdan qaysi amallarga (ko‘paytirish yoki bo‘lish) mos keladi? Misol orqali tushuntiring.
7. `np.packbits()` va `np.unpackbits()` funksiyalari nima uchun ishlatiladi?
Ularning xotirani tejashdagi ahamiyati nimada?
8. Yaxlitlash funksiyalari (`np.floor()` , `np.ceil()` , `np.trunc()`) o‘rtasidagi farq nima? Manfiy sonlar misolida tushuntiring.

3-daraja: Amaliy tahlil va Translyatsiya (Broadcasting)

9. Broadcasting (Translyatsiya) mexanizmi nima va uning asosiy afzalligi nimada? U xotira bilan qanday ishlaydi?
10. Ikki massiv o‘zaro mos kelishi (Broadcasting shartlari) uchun qanday qoidalar javob berishi kerak?
11. `np.char.rfind()` funksiyasi qanday natija qaytaradi? Agar qidirilayotgan qism-matn topilmasa, natija nima bo‘ladi?
12. Ma’lumotlarni markazlashtirish (Centering Data) va normallashtirishda (Normalization) translyatsiya qanday yordam beradi? Bu jarayon mashinali o‘rganish uchun nima uchun muhim?